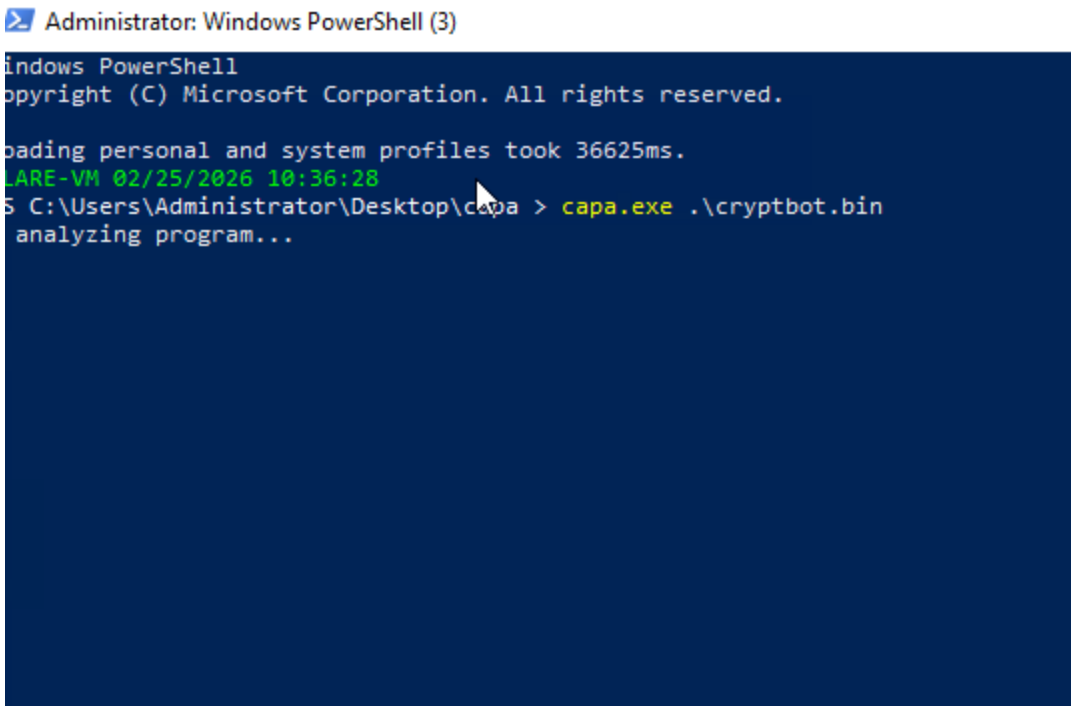
CAPA - The Basics

- CAPA (Common Analysis Platform for Artifacts) is a tool developed by the FireEye Mandiant team. It is designed to **identify the capabilities** present in executable files like Portable Executables (PE), ELF binaries, .NET modules, shellcode, and even sandbox reports.

Tool Overview: How CAPA Works

to run the tool you use capa.exe followed by the binary of the file you want to run the tool on :

```
capa.exe .\cryptbot.bin
```



Option	Description	Sample Syntax
`-h` or `--help`	Show this help message and exit.	`capa -h`
`-v` or `--verbose`	Enable verbose result document.	`capa.exe .\cryptbot.bin -v`
`-vv` or `--vverbose`	Enable a very verbose result document.	`capa.exe .\cryptbot.bin -vv`

```
md5                     | 3b9d26d2e7433749f2c32edb13a2b0a2
sha1                    | 969437df8f4ad08542ce8fc9831fc49a7765b7c5
sha256                  | ae7bc6b6f6ecb206a7b957e4bb86e0d11845c5b2d9f7a00a482bef63b567ce4c
analysis                | static
os                      | windows
format                  | pe
arch                    | i386
path                    | C:\Users\Administrator\Desktop\capa
```

```
ATT&CK Tactic           | ATT&CK Technique
DEFENSE EVASION         | Obfuscated Files or Information T1027
                        | Obfuscated Files or Information::Indicator Removal from Tools T1027.005
                        | Virtualization/Sandbox Evasion::System Checks T1497.001
DISCOVERY               | File and Directory Discovery T1083
EXECUTION               | Command and Scripting Interpreter::PowerShell T1059.001
                        | Shared Modules T1129
IMPACT                  | Resource Hijacking T1496
PERSISTENCE             | Scheduled Task/Job::At T1053.002
                        | Scheduled Task/Job::Scheduled Task T1053.005
```

```
MAEC Category               | MAEC Value
malware-category            | launcher
```

```
MBC Objective               | MBC Behavior
ANTI-BEHAVIORAL ANALYSIS    | Virtual Machine Detection [B0009]
ANTI-STATIC ANALYSIS        | Executable Code Obfuscation::Argument Obfuscation [B0032.020]
                            | Executable Code Obfuscation::Stack Strings [B0032.017]
```

Dissecting CAPA Results Part 1: General Information, MITRE and MAEC

The first block contains basic information about the file. This includes the following:

- The cryptographic algorithms, such as the `md5`, and `sha1/256`.
- The `analysis` field tells us how CAPA performed its analysis on the file.
- The `os` field reveals the operating system (OS) context for which the identified capabilities apply.
- The `arch` field allows us to determine whether we are dealing with a binary related to x86 architecture.
- The `path` where the analyzed file was located.

```
md5          | 3b9d26d2e7433749f2c32edb13a2b0a2
sha1         | 969437df8f4ad08542ce8fc9831fc49a7765b7c5
sha256       | ae7bc6b6f6ecb206a7b957e4bb86e0d11845c5b2d9f7a00a482bef63b567ce4c
analysis     | static
os           | windows
format       | pe
arch         | i386
path         | C:\Users\Administrator\Desktop\capa
```

MITRE ATT&CK

(Adversarial Tactics, Techniques, and Common Knowledge) framework is a comprehensive global knowledge repository that meticulously documents the tactics and techniques employed by threat actors at every stage of a cyber-attack

CAPA uses this format for the output. Note that some results may or may not contain the Technique and Sub-technique Identifier.

Format	Sample	Explanation
ATT&CK Tactic::ATT&CK Technique::Technique Identifier	Defense Evasion::Obfuscated Files or Information::T1027	**DEFENSE EVASION** = ATT&CK Tactic **Obfuscated Files or Information** = ATT&CK Technique **T1027** = Technique Identifier
ATT&CK Tactic::ATT&CK Technique::ATT&CK Sub-Technique::Technique Identifier[.]Sub-technique **Identifier**	Defense Evasion::Obfuscated Files or Information::Indicator Removal from Tools T1027.005	**DEFENSE EVASION** = ATT&CK Tactic **Obfuscated Files or Information** = ATT&CK Technique **Indicator Removal from Tools** = ATT&CK Sub-Technique **T1027** = Technique Identifier **005** = Sub-Technique Identifier

ATT&CK Tactic	ATT&CK Technique
DEFENSE EVASION	Obfuscated Files or Information T1027
	Obfuscated Files or Information::Indicator Removal from Tools T1027.005
	Virtualization/Sandbox Evasion::System Checks T1497.001
DISCOVERY	File and Directory Discovery T1083
EXECUTION	Command and Scripting Interpreter::PowerShell T1059.001
	Shared Modules T1129
IMPACT	Resource Hijacking T1496
PERSISTENCE	Scheduled Task/Job::At T1053.002
	Scheduled Task/Job::Scheduled Task T1053.005

MAEC

Malware Attribute Enumeration and Characterization is a specialized language designed to encode and communicate complex details concerning malware

Let's check the table below to see the most commonly used MAEC values by CAPA: **Downloader** and **Launcher**.

MAEC Value	Description
Launcher	Exhibits behaviours that trigger specific actions similar to malware behaviour.
Downloader	Exhibits behaviours wherein it downloads and executes other files, usually seen on more complex malware.

MAEC Category	MAEC Value
malware-category	launcher

When CAPA tags a file with a "**launcher**" MAEC value, it indicates that the file demonstrates behaviour similar to but not limited to:

- Dropping additional payloads
- Activating persistence mechanisms
- Connecting to command-and-control (C2) servers
- Executing specific functions

Dissecting CAPA Results Part 2: Malware Behaviour Catalogue

Malware Behaviour Catalogue (MBC)

MBC is designed to support various aspects of malware analysis, such as labelling, similarity analysis, and standardised reporting. Essentially, it serves as a catalogue of malware objectives and behaviours.

The content of MBC below can be represented in two formats.

Format	Sample	Explanation
OBJECTIVE::Behavior::Method[Identifier]	ANTI-STATIC ANALYSIS::Executable Code Obfuscation::Argument Obfuscation [B0032.020]	**Anti-static Analysis** = OBJECTIVE **Executable Code Obfuscation** = BEHAVIOR **Argument Obfuscation** = METHOD **BOO32.020** = IDENTIFIER
OBJECTIVE::Behavior::[Identifier]	COMMUNICATION::HTTP Communication:: [C0002]	**COMMUNICATION** = OBJECTIVE **HTTP Communication** = BEHAVIOR **C0002** = IDENTIFIER

The difference between the two formats is that the first format contains additional details called METHOD, which can also be coined as a sub-technique.

MBC Objective	MBC Behavior
ANTI-BEHAVIORAL ANALYSIS	Virtual Machine Detection [B0009]
ANTI-STATIC ANALYSIS	Executable Code Obfuscation::Argument Obfuscation [B0032.020] Executable Code Obfuscation::Stack Strings [B0032.017]
COMMUNICATION	HTTP Communication [C0002] HTTP Communication::Read Header [C0002.014]
DATA	Check String [C0019] Encode Data::Base64 [C0026.001] Encode Data::XOR [C0026.002]
DEFENSE EVASION	Obfuscated Files or Information::Encoding-Standard Algorithm [E1027.m02]
DISCOVERY	File and Directory Discovery [E1083]
EXECUTION	Command and Scripting Interpreter [E1059]
FILE SYSTEM	Create Directory [C0046] Delete File [C0047] Read File [C0051] Writes File [C0052]
MEMORY	Allocate Memory [C0007]
PROCESS	Create Process [C0017]

Objective

The Objective are based on **ATT&CK tactics in the context of malware behaviour**, though not all are included. Furthermore, MBC has Anti-Behavioral and Anti-Static Analysis. These objectives are tailored for malware analysis with the use case of **characterising malware**

Objective	Explanation
Anti-Behavioral Analysis	Malware attempts to avoid detection by hindering behavioural analysis using tools like sandboxes or debuggers.
Anti-Static Analysis	Malware attempts to obstruct or add complexity to static analysis, making it more challenging for security professionals to identify and understand its malicious behaviours and intentions.
Collection	Malware focuses on identifying and gathering information from the targeted machine or network.
Command and Control	Malware typically establishes communication with compromised systems through various methods such as command and control servers, peer-to-peer networks, or other means. This communication allows the malware to control the compromised systems, enabling the attackers to execute commands, exfiltrate data, or carry out other malicious activities.
Credential Access	The primary aim of malware is to steal account credentials, such as usernames and passwords.
Defense Evasion	The malware aims to bypass and circumvent the various detection and security mechanisms present within the system to avoid being detected or mitigated.

Discovery	Malware Seeks to collect detailed information about the configuration and setup of the system or network environment, including hardware, software, and network infrastructure.
Execution	Malware is designed to execute unauthorized commands or code on a targeted computer system without the user's consent. This can include a wide range of harmful activities, such as stealing personal information, damaging files, or gaining unauthorized access to the system.
Exfiltration	Malware is designed to infiltrate computer systems or networks to steal and extract sensitive data. This can include personal information, financial details, and any other valuable data stored on the targeted system or network.
Impact	Malware aims to manipulate, disrupt, or damage computer systems and data. It can enter computers through infected emails, compromised websites, and other deceptive means, leading to financial loss, privacy breaches, and system instability.
Lateral Movement	Malware seeks to spread through the network, either actively through machine access or passively, such as via malicious emails.
Persistence	Malware is intentionally developed with the capability to remain undetected and operational on a computer system for an extended period.
Privilege Escalation	Malware seeks to infiltrate a computer system or network to gain elevated permissions or control. Once inside the target environment, malware can seek to escalate its privileges, access sensitive information, or take control of system resources for malicious purposes.

Micro-Objective

Micro-objectives are associated with micro-behaviors, which refer to an action or actions exhibited by potentially malicious software that isn't necessarily malicious and may serve various objectives.

Micro-Objective	Description
PROCESS	exhibiting behaviours related to processes such as but not limited to Creating Process, Setting Thread Context, Terminating Process, and Checking Mutex.
MEMORY	exhibiting behaviours such as, but not limited to, Allocating Memory, Changing Memory Protection, and Freeing Memory.
COMMUNICATION	exhibiting behaviours such as (not limited to (DNS, FTP, HTTP, ICMP, SMTP) network traffic.
DATA	exhibiting behaviours such as but not limited to Checking strings, compressing, decoding and encoding data

MBC Behaviors

The column MBC Behaviors contains behaviours and Micro-behaviors with or without its methods and identifiers. Please check the link MBC Summary for a listing of all MBC content.

Objective	Behavior	Identifiers	Explanation
ANTI-BEHAVIORAL ANALYSIS	**Virtual Machine Detection**	**B0009**	The malware checks to see if it is running in a virtual environment. During its system reconnaissance, the malware examines various user and system artifacts.
ANTI-STATIC ANALYSIS	**Executable Code Obfuscation**	**B0032**	Executable code has been intentionally obscured to prevent static code analysis. This is a specific behavior related to the executable code of a malware sample, including its data and text sections.
EXECUTION	**Command and Scripting Interpreter**	**E1059**	Malware can exploit command and script interpreters to run malicious commands, scripts, or binaries. It targets built-in interpreters like cmd.exe or PowerShell on Windows or Bash on Unix-like systems. Attackers may also use other scripting languages like Python, Perl, or JavaScript.
DISCOVERY	**File and Directory Discovery**	**E1083**	Malware has the capability to search for specific files in particular locations by enumerating files and directories.
ANTI-STATIC ANALYSIS, DEFENSE EVASION	**Obfuscated Files or Information**	**E1027**	Malware can obfuscate files or information by encoding, encrypting, or otherwise, making them hard to analyze. It can also encode or encrypt malware samples itself.

Micro-Behavior

The term "low-level behaviors" in malware analysis refers to actions exhibited by malware that aren't necessarily malicious and may serve various objectives. These behaviors are often documented as "micro-behaviors" in the Malware Behavior Characteristics (MBC) analysis.

Micro-Objective	Micro-Behaviors	Identifiers	Explanation
MEMORY	**Allocate Memory**	**C0007**	Malware frequently utilizes memory allocation as part of its strategy to unpack itself and execute its malicious activities.
PROCESS	**Create Process**	**C0017**	Malware creates a process via WMI or shellcode. It can also create a suspended process.
COMMUNICATION	**HTTP Communication**	**C0002**	Malware is capable of initiating HTTP communications.
DATA	**Check String**	**C0019**	Malware can inspect a string to identify specific characteristics, such as ASCII content, credit card numbers, and string length.
DATA	**Encode Data**	**C0026**	Malware has the capability to encode data using base64 and XOR.
FILE SYSTEM	**Create Directory**	**C0046**	Malware can create a directory.
FILE SYSTEM	**Delete File**	**C0047**	Malware has the capability to delete a file.
FILE SYSTEM	**Read File**	**C0051**	Malware can read a file.
FILE SYSTEM	**Writes File**	**C0052**	Malware has the capability to write to a file.

Methods

Methods are tied to behaviors; therefore, to fully see all methods, please refer to each specific behavior/micro behavior of interest.

Behavior	Methods or sub-technique	Identifier	Explanation
Executable Code Obfuscation	**Argument Obfuscation**	B0032.020	Simple number or string arguments to API calls are calculated at runtime, making analysis more difficult.
Executable Code Obfuscation	**Stack Strings**	B0032.017	Build and decrypt strings on the stack at each use, then discard to avoid obvious references.
HTTP Communication	**Read Header**	C0002.014	HTTP read header.
Encode Data	**Base64**	C0026.001	Malware may encode data using Base64.
Encode Data	**XOR**	C0026.002	Malware may use XOR to encode data.
Obfuscated Files or Information	**Encoding-Standard Algorithm**	E1027.m02	Encoding malware samples, files, or other information uses a standard algorithm (e.g., base64).

Dissecting CAPA Results Part 3: Namespaces

```
-----------------------+-------------------------------------------------------------------------------
Capability                                                    | Namespace
-----------------------+-------------------------------------------------------------------------------
reference anti-VM strings                                     | anti-analysis/anti-vm/vm-detection
reference anti-VM strings targeting VMWare                    | anti-analysis/anti-vm/vm-detection
reference anti-VM strings targeting VirtualBox                | anti-analysis/anti-vm/vm-detection
contain obfuscated stackstrings (2 matches)                   | anti-analysis/obfuscation/string/stackstring
reference HTTP User-Agent string                              | communication/http
check HTTP status code                                        | communication/http/client
reference Base64 string                                       | data-manipulation/encoding/base64
encode data using XOR                                         | data-manipulation/encoding/xor
contain a thread local storage (.tls) section                 | executable/pe/section/tls
get common file path                                          | host-interaction/file-system
create directory                                              | host-interaction/file-system/create
delete file                                                   | host-interaction/file-system/delete
read file on Windows (4 matches)                              | host-interaction/file-system/read
write file on Windows (5 matches)                             | host-interaction/file-system/write
get thread local storage value                                | host-interaction/process
create process on Windows                                     | host-interaction/process/create
allocate or change RWX memory                                 | host-interaction/process/inject
reference cryptocurrency strings                              | impact/cryptocurrency
link function at runtime on Windows (5 matches)               | linking/runtime-linking
parse PE header (4 matches)                                   | load-code/pe
resolve function by parsing PE exports (186 matches)          | load-code/pe
run PowerShell expression                                     | load-code/powershell/
schedule task via at                                          | persistence/scheduled-tasks
schedule task via schtasks                                    | persistence/scheduled-tasks
```

Format	Sample	Explanation
Capability(Rule Name)::TLN(Top-Level Namespace)/Namespace	reference anti-VM strings::Anti-Analysis/anti-vm/vm-detection	**Reference anti-VM strings** = Capability(Rule Name) **Anti-Analysis** = TLN or Top-Level Namespace **anti-vm/vm-detection** = Namespace

Namespaces

CAPA uses namespaces to group items with the same purpose.

Top-Level Namespace (TLN)	Explanation
anti-analysis	contains a set of rules specifically designed to detect behaviours exhibited by malware to evade analysis. These behaviours include obfuscation, packing, and anti-debugging techniques.
collection	contains a set of data-related rules that malware may enumerate and collect for exfiltration or other purposes. Think of it as the “data-gathering” aspect of malware behaviour.
communication	contains a set of rules that pertain to different communication behaviours demonstrated by malware. This encompasses how malware interacts with networks, including data transmission and reception, command and control communications, and other network-related behaviours.
compiler	contains a set of rules and configurations for recognizing specific build environments or compilers employed in generating executables. These namespaces essentially serve as the unique “signature” that identifies the compilation process of a program.
data-manipulation	contains a set of rules that govern the behaviours involved in altering data within executable files. This aspect can be considered the “data transformation” component of malware behaviour, encompassing actions such as String Encryption and Data Encoding.

executable	contains a set of rules pertaining to the attributes in executable files. These attributes include PE sections or debug info associated with the executable.
host-interaction	contains a set of rules related to behaviors involving interactions with the host system. This encompasses how malware interacts with its environment. Specifically, the rules in this namespace may capture behaviors related to reading, writing, or modifying files on disk, including creating, deleting, or modifying files and directories."
impact	contains a set of rules related to the potential consequences or effects of a program's behavior. Think of it as the aspect that focuses on the possible harm that this malware can cause. It may include behaviors related to establishing remote access, data exfiltration, destruction, or modification."
internal	Rules contained within the system are not intended for direct use by analysts or for reporting. Instead, these rules are meant for internal purposes within the CAPA tool, serving as the behind-the-scenes aspect of rule development and execution.
lib	building blocks to create other rules
linking	contains rules to identify behaviors involving linking or dynamically loading external code or libraries during program execution. This is its primary function and is crucial for the program's security. Understanding linking behavior is essential for several reasons. Malware often depends on external libraries or components (such as OpenSSL, Zlib, or other third-party libraries) to carry out specific tasks. Detecting these dependencies helps analysts understand the malware's capabilities. External libraries also introduce an additional attack surface. If a vulnerability exists in a linked library, it can be exploited by the malware or defenders during analysis.

load-code	contains a set of rules and regulations related to the behaviors associated with dynamically loading or executing code during program execution. This concept can be equated to the "runtime code injection" aspect of malware behavior, which involves unauthorized code introduction during a program's execution.
malware-family	contains a set of rules related to behaviors that are linked to particular malware families or groups. It serves as a way to identify the distinct characteristics or "signatures" associated with known malware families, allowing for more accurate detection and classification of potential threats.
nursery	this is a staging ground that contains rules for those who are not quite polished
persistence	contains rules related to behaviors associated with maintaining access or persistence within a compromised system. This namespace is essentially focused on understanding how malware can establish and maintain a presence within a compromised environment, allowing it to persist and carry out malicious activities over an extended period.
runtime	contains a set of rules that seek to identify the language or platform on which the program runs.
targeting	contains a set of rules related to behaviors exhibited by programs that interact with ATMs.

example:

Top-Level Namespace (TLN)	Namespaces	Rule YAML File	Explanation
Anti-Analysis	anti-vm/vm-detection	reference-anti-vm-strings-targeting-virtualbox.yml reference-anti-vm-strings-targeting-virtualpc.yml	"anti-vm/vm-detection" namespace contains rules to detect virtual machine (VM) environments. These rules focus on identifying specific strings or patterns commonly used by malware to detect VMs while running. Using these rules, CAPA can identify if malware searches for VMware-specific registry keys, the presence of VMware tools, or other VM-related elements.
	obfuscation	obfuscated-with-dotfuscator.yml obfuscated-with-smartassembly.yml	Malware often uses obfuscation techniques to make analysis more difficult. These include methods such as String Encryption, Code Obfuscation, Packing, and Anti-Debugging Tricks. The obfuscation namespace addresses these techniques, which conceal or obscure the true purpose of the code.

Dissecting CAPA Results Part 4: Capability

Capability and its related TLN, namespace, and the rules associated with the yaml file.

Capability	Top-Level Namespace (TLN)	Namespaces	Rule YAML file	Notes
reference anti-VM strings	**Anti-Analysis**	anti-vm/vm-detection	reference-anti-vm-strings.yml	To check all rules under this namespace, click here
reference anti-VM strings targeting VMWare	**Anti-Analysis**	anti-vm/vm-detection	reference-anti-vm-strings-targeting-vmware.yml	To check all rules under this namespace, click here
reference anti-VM strings targeting VirtualBox	**Anti-Analysis**	anti-vm/vm-detection	reference-anti-vm-strings-targeting-virtualbox.yml	You may check the TLN(Top-Level Namespace).
reference HTTP User-Agent string	**Communication**	http/client	reference-http-user-agent-string.yml	To check all rules under this namespace, click here)
check HTTP status code	**Communication**	http	check-http-status-code.yml	To check all rules under this namespace, click here

reference Base64 string	**Data Manipulation**	encoding/base64	reference-base64-string.yml	To check all rules under this namespace, click here
encode data using XOR	**Data Manipulation**	encoding/XOR	encode-data-using-xor.yml	To check all rules under this namespace, click here
contain a thread local storage (.tls) section	**Executable**	pe/section/tls	contain-a-thread-local-storage-tls-section.yml	You may check the TLN(Top-Level Namespace) for more rules.
get common file path	**Host-Interaction**	file-system	get-common-file-path.yml	You may check the TLN(Top-Level Namespace) for more rules.
create directory	**Host-Interaction**	file-system/create	create-directory.yml	You may check the TLN(Top-Level Namespace) for more rules.
delete file	**Host-Interaction**	file-system/delete	delete-file.yml	To check all rules under this namespace, click here
read file on Windows	**Host-Interaction**	file-system/read	read-file-on-windows.yml	To check all rules under this namespace, click here

get thread local storage value	**Host-Interaction**	process	get-thread-local-storage-value.yml	This rule is found under **TLN Nursery,** a staging ground for unpolished rules.
allocate or change RWX memory	**Host-Interaction**	process/inject	allocate-or-change-rwx-memory.yml	To check all rules under this namespace, click here
create process on Windows	**Host-Interaction**	process create	create-process-on-windows.yml	To check all rules under this namespace, click here
reference cryptocurrency strings	**Impact**	impact/cryptocurrency	reference-cryptocurrency-strings.yml	This rule is found under **TLN Nursery,** a staging ground for unpolished rules.
link function at runtime on Windows	**Linking**	runtime-linking	link-function-at-runtime-on-windows.yml	To check all rules under this namespace, click here
parse PE header	**load-code**	load-code/pe	parse-pe-header.yml resolve-function-by-parsing-pe-exports.yml	To check all rules under this namespace, click here

resolve function by parsing PE exports	**load-code**	load-code/pe	resolve-function-by-parsing-pe-exports.yml	To check all rules under this namespace, click here
run PowerShell expression	**load-code**	load-code/PowerShell	run-powershell-expression.yml	To check all rules under this namespace, click here
schedule task via at	**persistence**	scheduled-tasks	schedule-task-via-at.yml	You may check the TLN(Top-Level Namespace) for more rules.
schedule task via schtasks	**persistence**	scheduled-tasks	schedule-task-via-schtasks.yml	You may check the TLN(Top-Level Namespace) for more rules.

More Information, more fun!

https://github.com/MBCProject/capa-rules-1?tab=readme-ov-file#namespace-organization

determine the reason for triggering the rules and the conditions involved

Option	Description	Sample Syntax
`-v` or `--verbose`	enable verbose result document	`capa.exe -v .\cryptbot.bin`
`-vv` or `--vverbose`	enable very verbose result document)	`capa.exe -vv .\cryptbot.bin`